EXPERIMENT - 62

1. Problem Statement

Write a C program that reads the sizes and elements of two sorted variable-length integer arrays, arr1[] and arr2[], containing their elements in ascending order. Pass both arrays to a function named merge_sorted_arrays() using their addresses. Use pointers to pass the arrays between the calling function and merge_sorted_arrays(). The function merge_sorted_arrays() must:

- Merge arr1[] and arr2[] into a third array, merged[].
- Maintain the ascending order of the elements in merged[].
- Preserve duplicate elements, if any.
- Display the elements of merged[] separated by spaces.
- Use the two-pointer technique to merge the arrays efficiently.

2. Algorithm

Main Program

Step 1: START
Step 2: Read n1, n2 and the elements of arr1 and arr2 (variable length arrays).
Step 3: Declare merged of size n1 + n2.
Step 4: Call merge_sorted_arrays(&arr1[0], n1, &arr2[0], n2, &merged[0]).
Step 5: STOP

Function merge_sorted_arrays()

Step 1: Set i = 0, j = 0, k = 0.
Step 2: While i < n1 and j < n2: if arr1[i] <= arr2[j], copy arr1[i] to merged[k] and increase i, else copy arr2[j] and increase j. Increase k each time.
Step 3: Copy the remaining elements of arr1 or arr2 to merged.
Step 4: Display the n1 + n2 elements of merged separated by spaces and return.

3. Test Case Table

#	Array 1	Array 2	Expected Output	Actual Output	Result
1	1 3 5 7	2 4 6 8	1 2 3 4 5 6 7 8	1 2 3 4 5 6 7 8	PASS
2	1 2 2 4	2 3 5	1 2 2 2 3 4 5, duplicate 2 kept	1 2 2 2 3 4 5	PASS
3	1 4 7	2 3 5 6 8	1 2 3 4 5 6 7 8	1 2 3 4 5 6 7 8	PASS
4	7 8 9	1 2	1 2 7 8 9, all of arr2 is smaller	1 2 7 8 9	PASS
5	2 2 2	2 2 2	2 2 2 2 2 2, all elements equal	2 2 2 2 2 2	PASS

4. Source Code

merge62.c

```
/* Experiment 62: merge two sorted arrays using two pointers. */
#include <stdio.h>

void merge_sorted_arrays(int *arr1, int n1, int *arr2, int n2, int *merged);

int main(void)
{
    int n1, n2, i;

    printf("Enter the size of array 1: ");
    scanf("%d", &n1);
    int arr1[n1];
    printf("Enter %d elements: ", n1);
    for (i = 0; i < n1; i++)
        scanf("%d", &arr1[i]);

    printf("Enter the size of array 2: ");
    scanf("%d", &n2);
    int arr2[n2];
    printf("Enter %d elements: ", n2);
    for (i = 0; i < n2; i++)
        scanf("%d", &arr2[i]);

    int merged[n1 + n2];
    merge_sorted_arrays(&arr1[0], n1, &arr2[0], n2, &merged[0]);
    return 0;
}

/* i walks arr1, j walks arr2, k fills merged. "<=" keeps duplicates. */
void merge_sorted_arrays(int *arr1, int n1, int *arr2, int n2, int *merged)
{
    int i = 0, j = 0, k = 0;

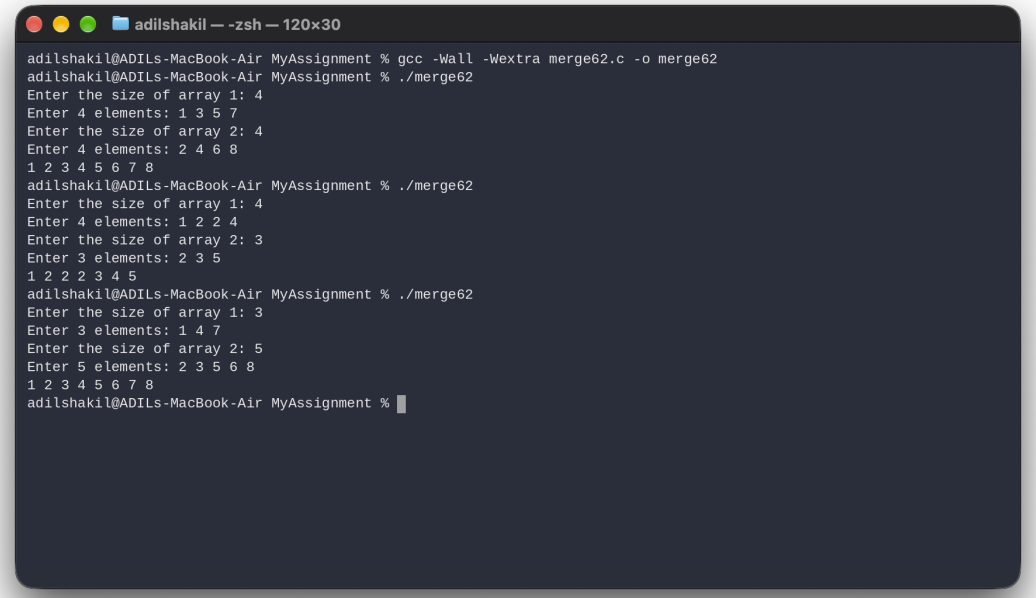
    while (i < n1 && j < n2) {
        if (*(arr1 + i) <= *(arr2 + j))
            *(merged + k++) = *(arr1 + i++);
        else
            *(merged + k++) = *(arr2 + j++);
    }
    while (i < n1)
        *(merged + k++) = *(arr1 + i++);
    while (j < n2)
        *(merged + k++) = *(arr2 + j++);

    for (k = 0; k < n1 + n2; k++)
        printf("%d ", *(merged + k));
    printf("\n");
}
```

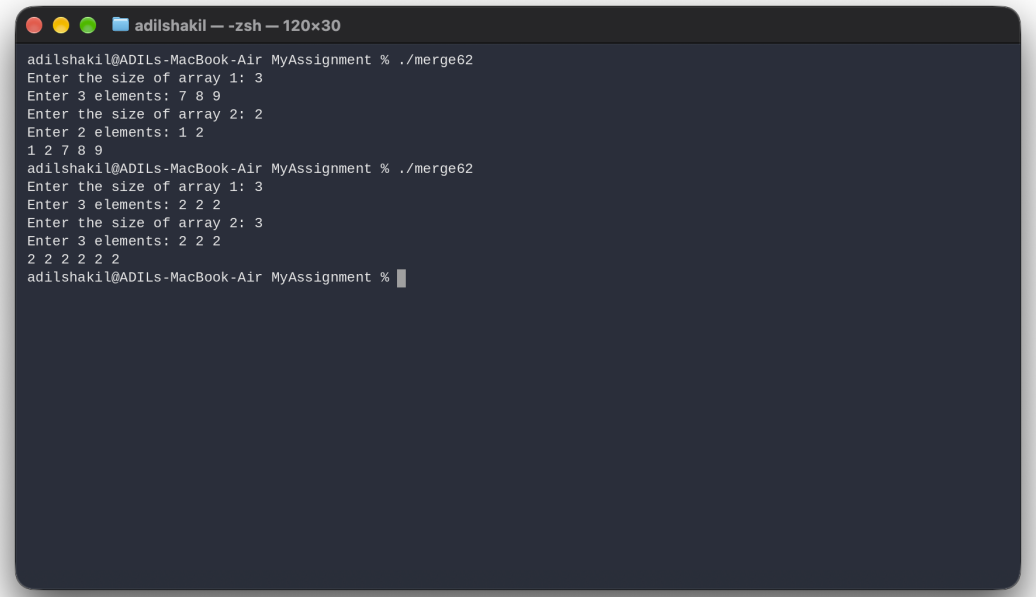
5. Compilation and Output

Compiled with `gcc -Wall -Wextra merge62.c -o merge62` so the small warnings would show up too. It built clean.

Cases 1 to 3



Cases 4 and 5



All five passed, and the first three are the examples from the question.